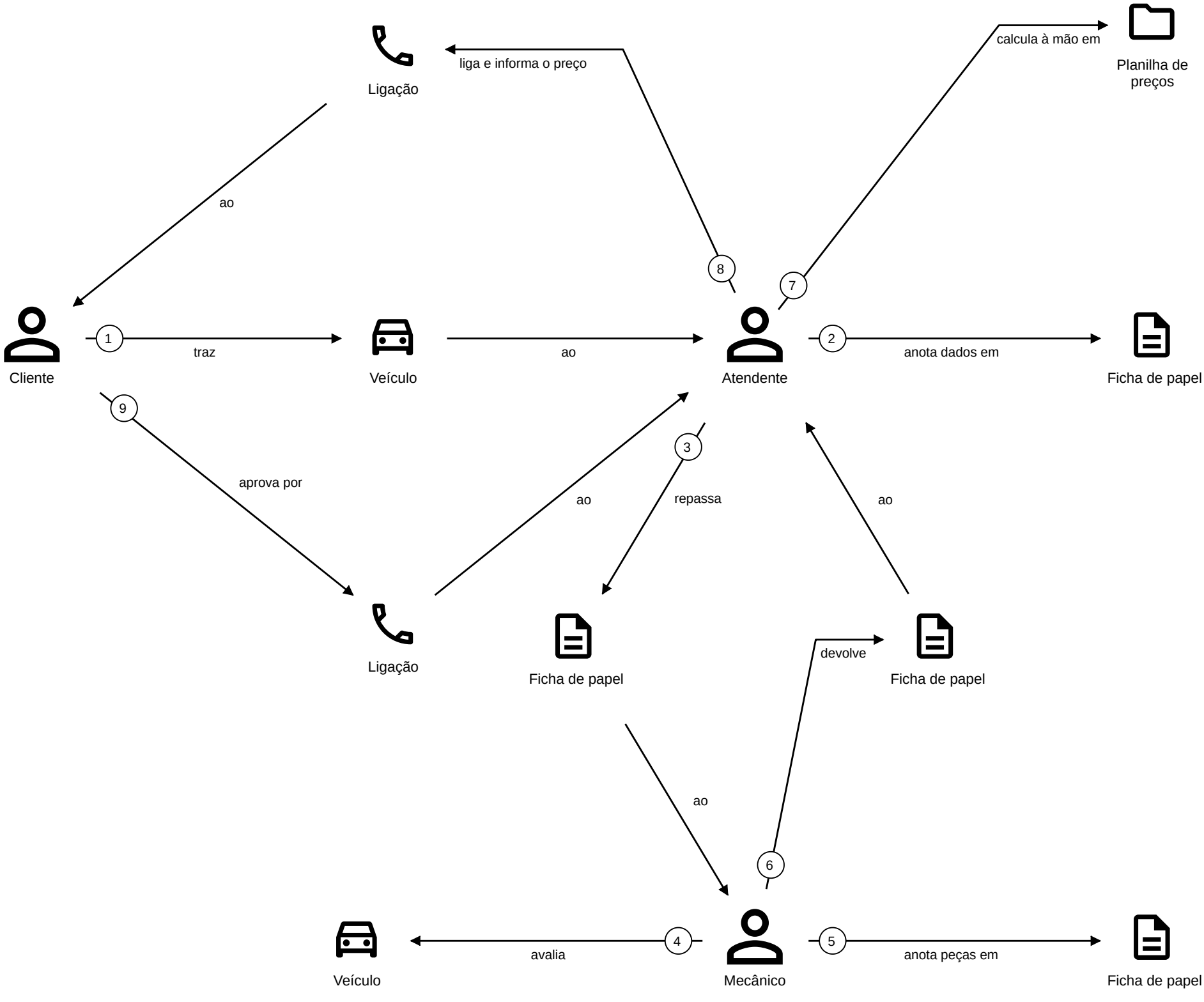
Domain Storytelling · As-Is 1 · Chegada e orçamento

as-is-1-chegada-orcamento

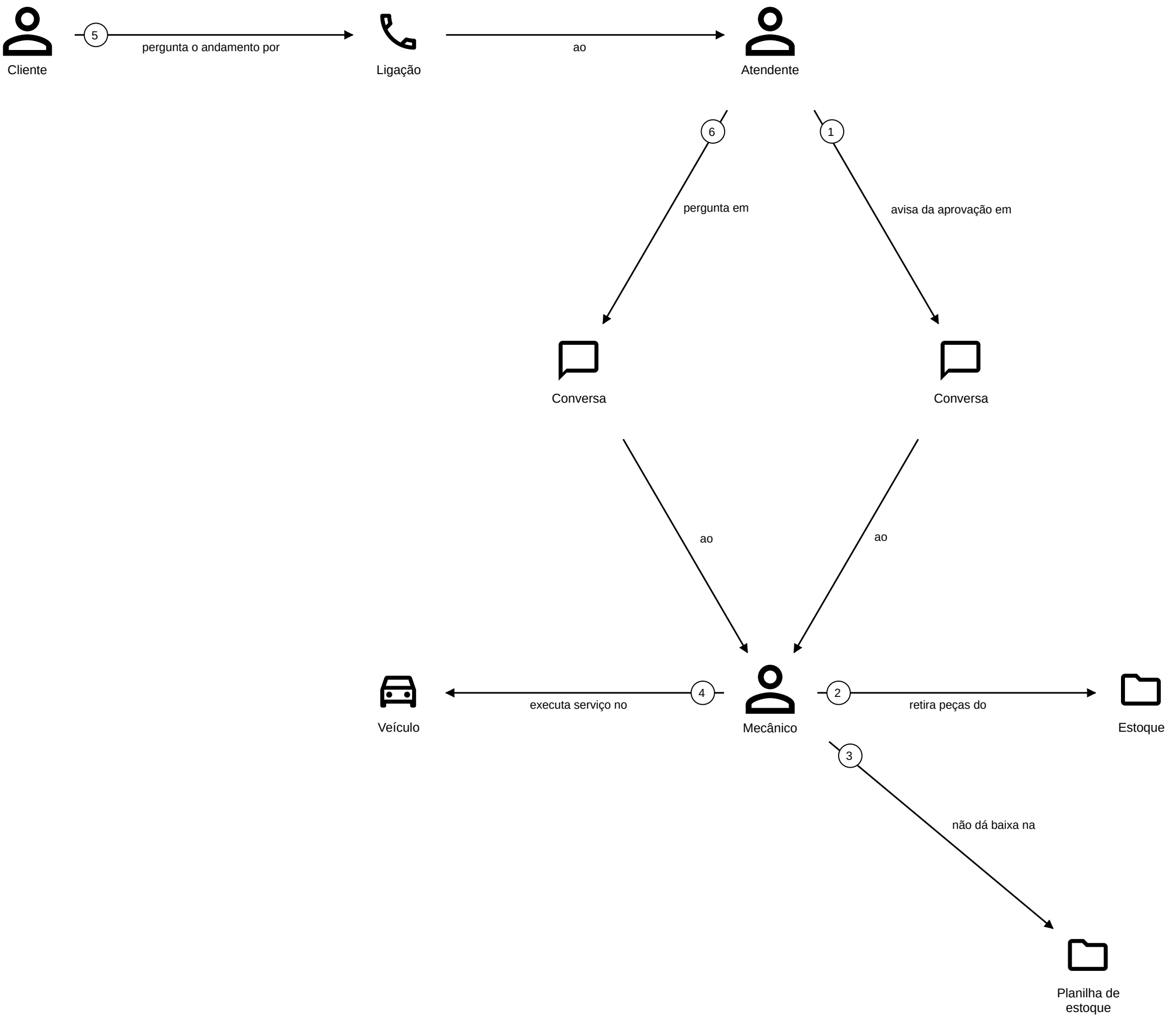
As-Is · Chegada e orçamento: como a oficina trabalha hoje, com papel, planilha e telefone.



Domain Storytelling · As-Is 2 · Execução e estoque

as-is-2-execucao-estoque

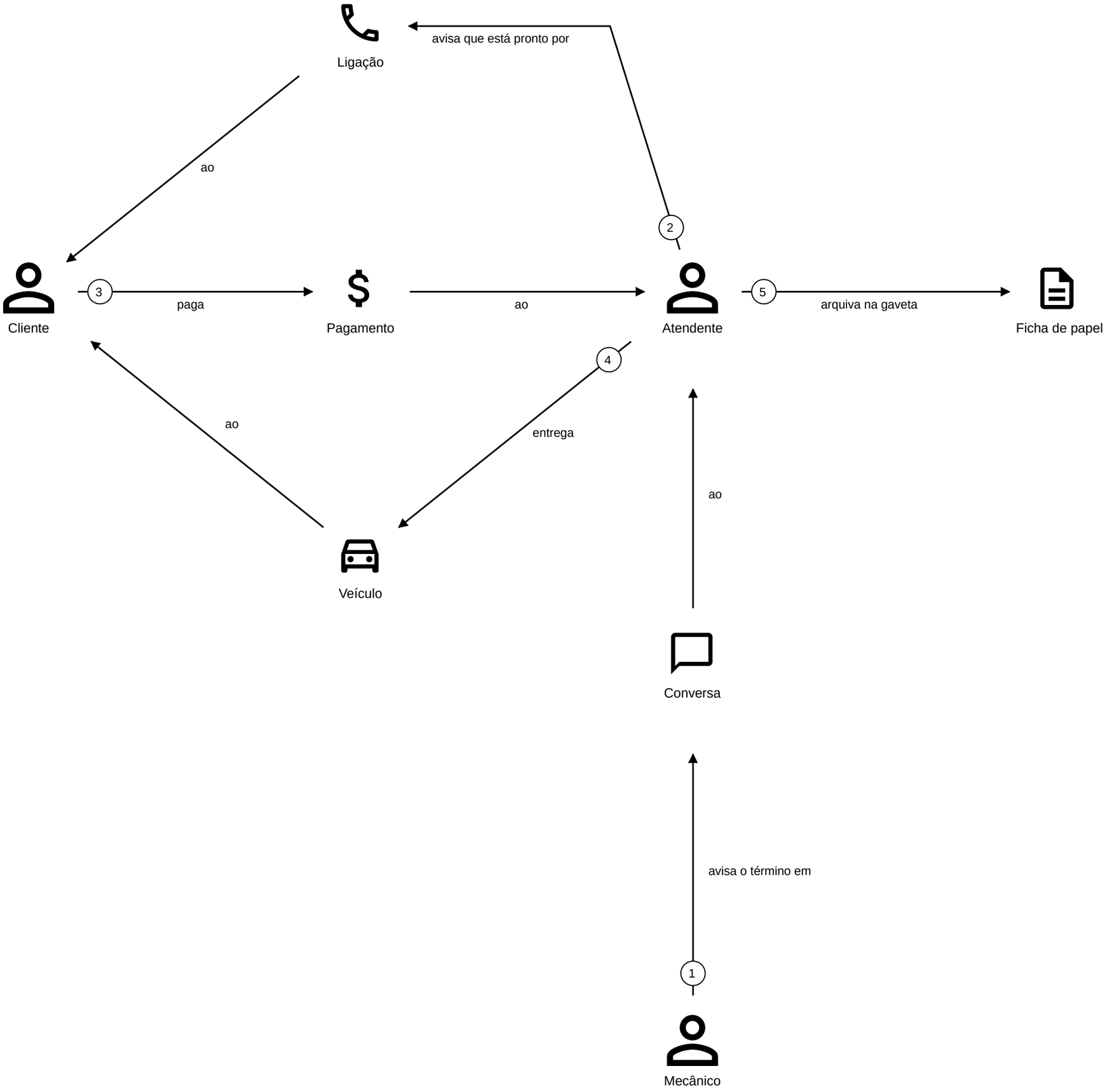
As-Is · Execução e estoque: aprovação de boca, peça retirada sem baixa e acompanhamento por telefone.



Domain Storytelling · As-Is 3 · Entrega

as-is-3-entrega

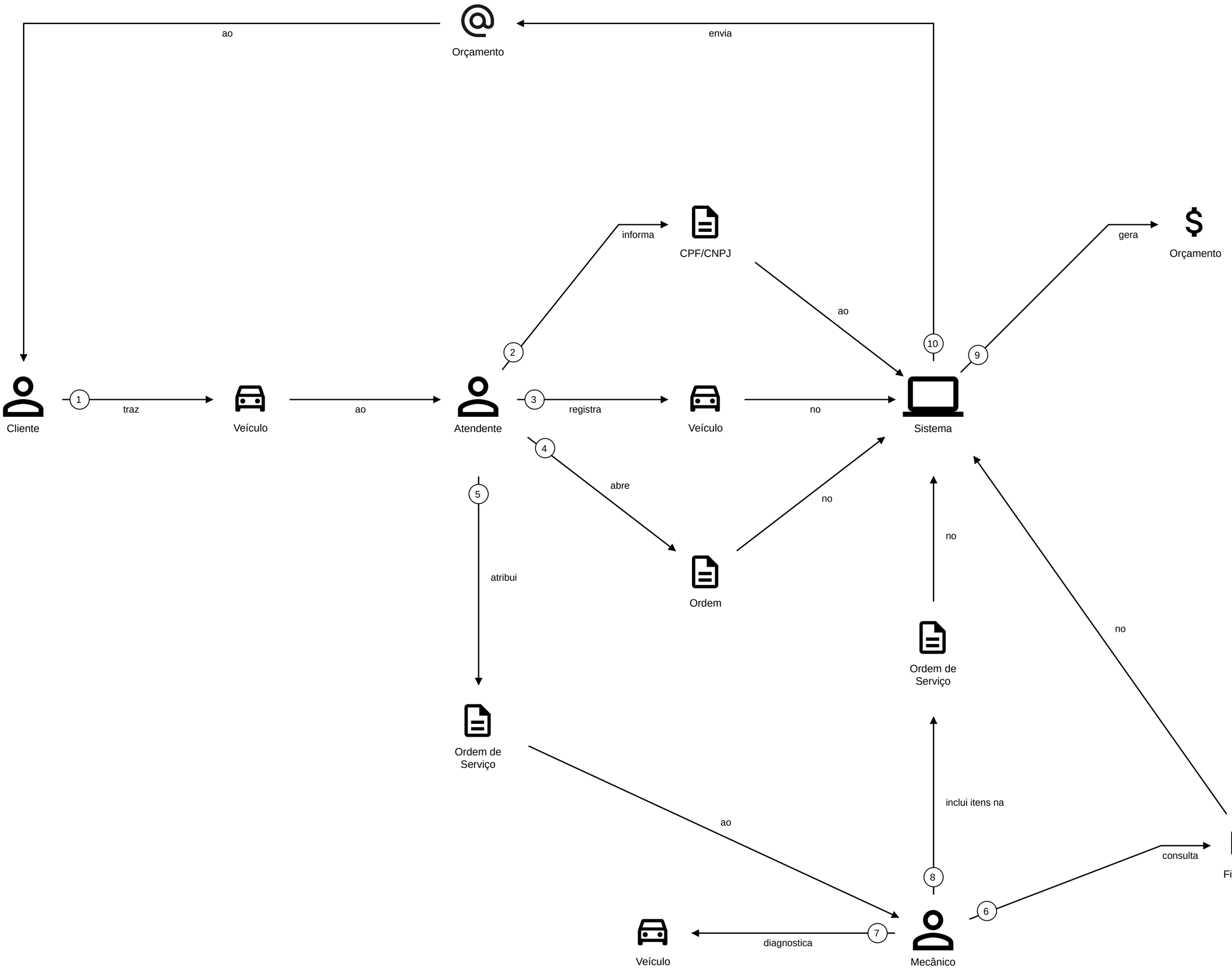
As-Is · Entrega: aviso manual, pagamento no balcão e ficha arquivada em papel.



Domain Storytelling · To-Be 1 · Criação da OS

to-be-1-criacao-os

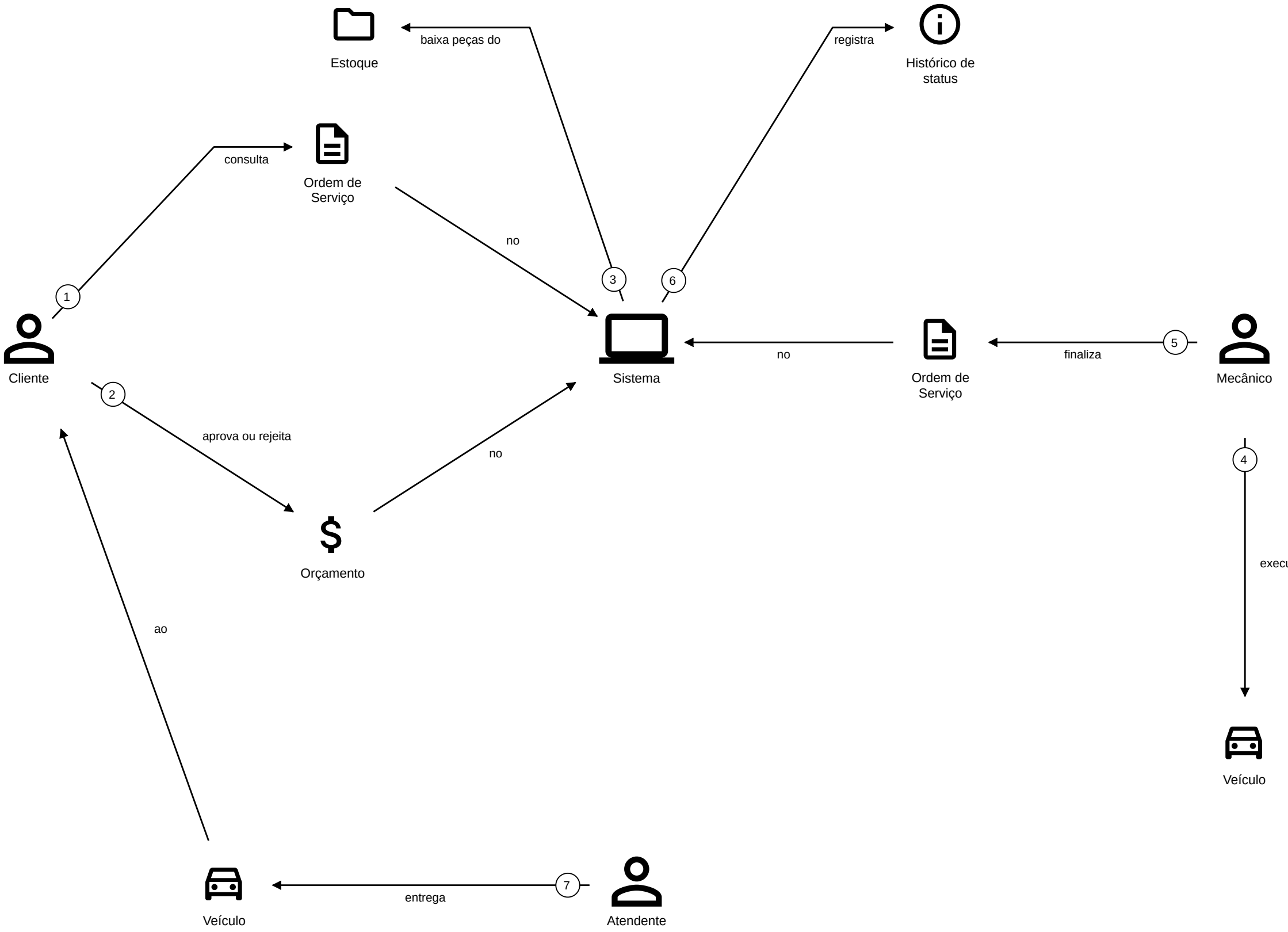
To-Be · Criação da OS: o atendimento é registrado no Sistema e o orçamento é gerado automaticamente.



Domain Storytelling · To-Be 2 · Acompanhamento

to-be-2-acompanhamento

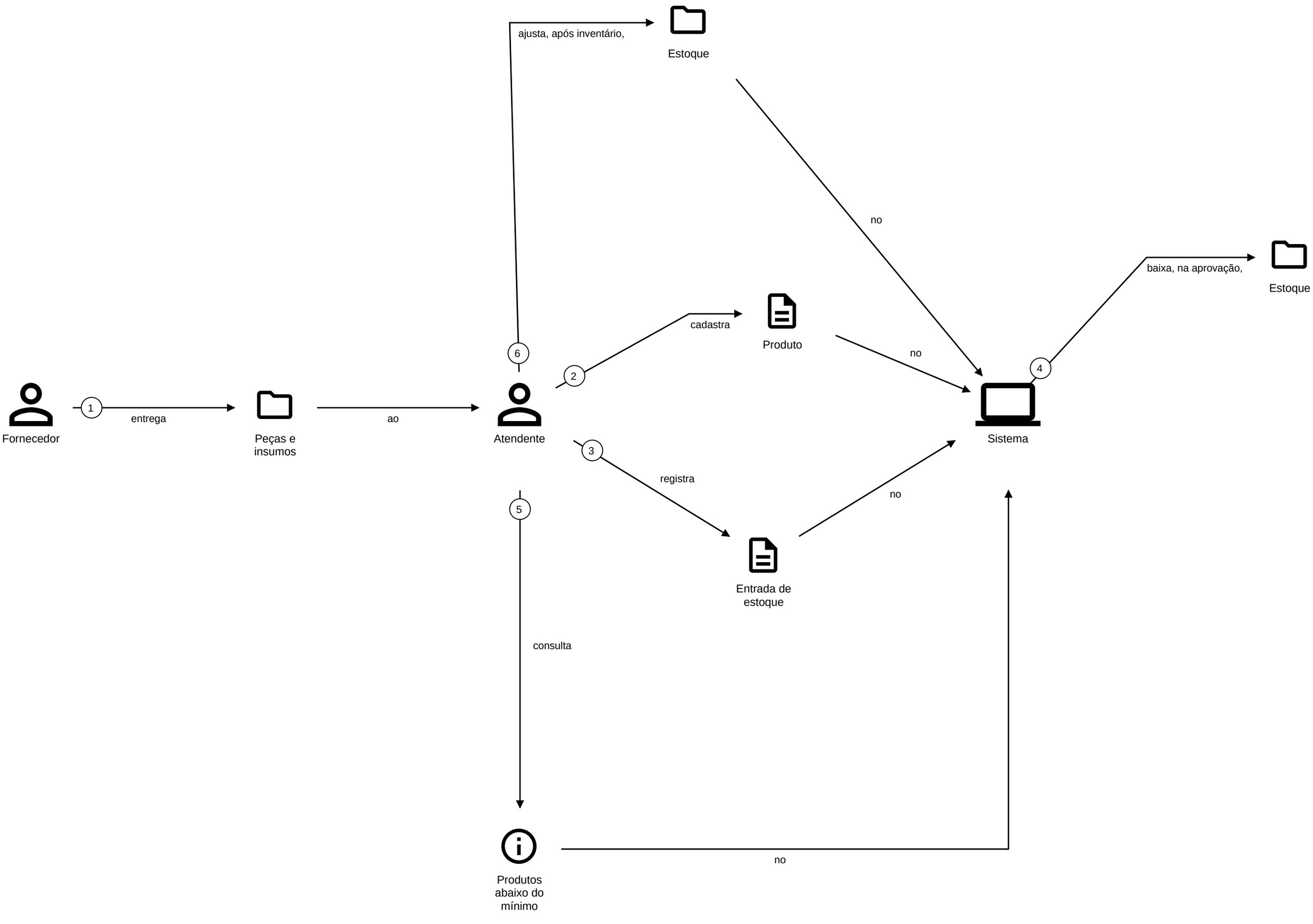
To-Be · Acompanhamento: o cliente decide e acompanha pelo Sistema; estoque e histórico são atualizados sozinhos.



Domain Storytelling · To-Be 3 · Peças e insumos

to-be-3-pecas-insumos

To-Be · Peças e insumos: todo saldo muda por uma movimentação registrada no Sistema.



Subdomínios — Manutenção e reparo de veículos

Classificação estratégica: onde está o diferencial da oficina e onde basta uma solução simples ou pronta

CORE — o diferencial do negócio

Gestão de Ordens de Serviço

Diagnóstico, orçamento, aprovação pelo cliente, execução e acompanhamento de status em tempo real.
Por quê: resolve as dores centrais da oficina e tem a lógica mais rica (transições de status, orçamento congelado, aprovação atômica com baixa de estoque).
Bounded context: Atendimento · módulo *service-order*

SUPORTE — necessário, mas não diferencia

Catálogo e Estoque

Serviços, peças e insumos, saldo e movimentações.
Por quê: necessário para o orçamento; regras simples (saldo ≥ 0, mínimo).
Bounded context: Catálogo e Estoque · *inventory*

Clientes e Veículos

Cadastro com validação de CPF/CNPJ e placa.
Por quê: essencialmente CRUD com validação; sem vantagem competitiva.
Bounded context: Clientes e Veículos · *customer, vehicle*

Indicadores operacionais

Tempo médio de execução dos serviços.
Por quê: cálculo derivado do histórico da OS, sem regra própria.
Bounded context: dentro de Atendimento (read model)

GENÉRICO — comum a qualquer empresa, pode ser comprado

Identidade e Acesso

Login, senha (bcrypt) e JWT com papéis.
Por quê: todo sistema tem; usamos bibliotecas prontas (jose, bcrypt).
Bounded context: Identidade e Acesso · *user, auth, employee*

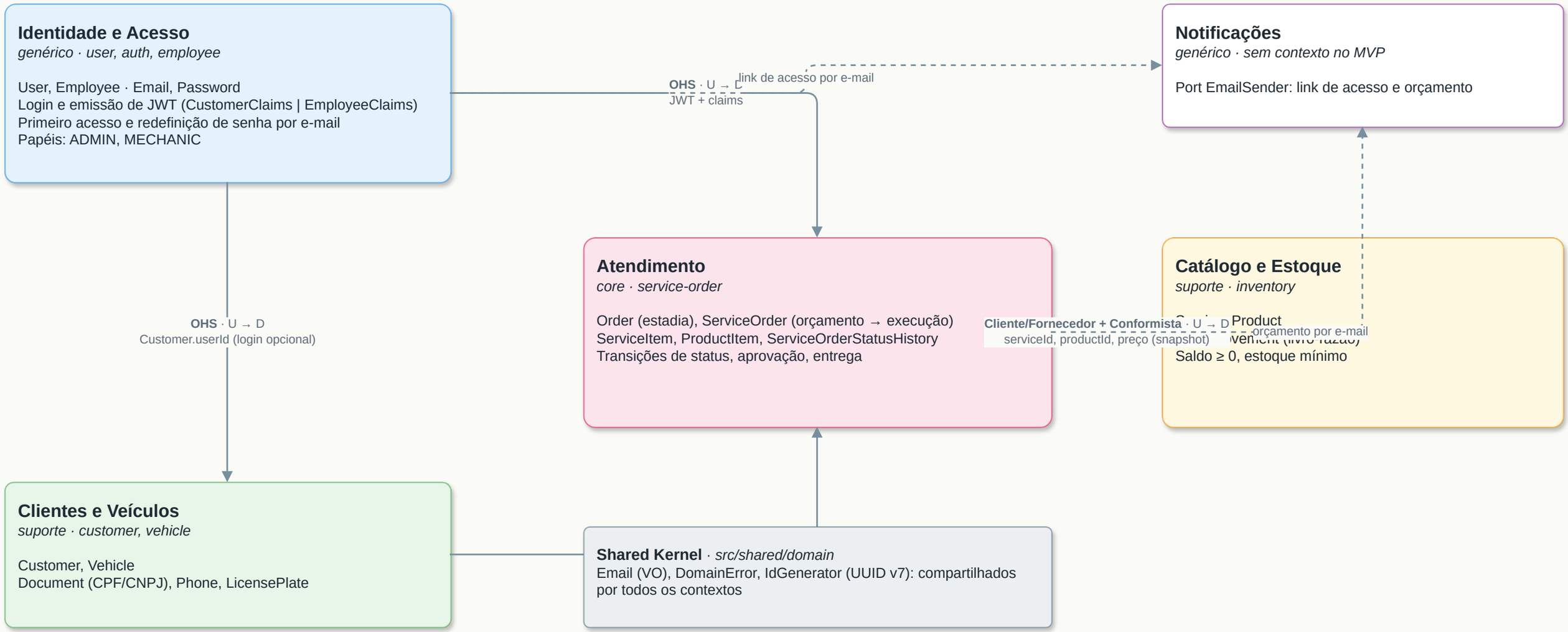
Notificações

Envio do link de acesso e do orçamento por e-mail.
Por quê: commodity (serviço de e-mail).
Bounded context: sem contexto próprio no MVP (port EmailSender usado por Identidade e Atendimento)

Critério (fluxograma de Vlad Khononov): a solução pode ser comprada ou adotada pronta sem risco para o negócio? → Genérico. Não pode, e a lógica é complexa? → Core. Não pode, mas a lógica é simples? → Suporte.

Context Map — bounded contexts e padrões de integração

Monolito modular: cada contexto é um conjunto de módulos em src/modules; U = upstream, D = downstream



- **OHS (Open Host Service):** Identidade expõe um contrato estável (token + claims) que os outros contextos só consomem, sem conhecer como a autenticação funciona.
- **Cliente/Fornecedor:** Atendimento depende dos dados de Clientes e do Catálogo; os upstreams evoluem considerando essa necessidade.
- **Conformista:** Atendimento usa o modelo do Catálogo como está (preço e item) e congela uma cópia na OS.
- **Sem ACL:** todos os contextos estão no mesmo monolito, com a mesma linguagem ubíqua e contratos internos por port; uma camada anticorrupção só faria sentido com um sistema externo ou legado.
- Relações sempre **por ID**: nenhum agregado navega para dentro de outro contexto.

Agregados, entidades e value objects

«Aggregate Root» «Entity» «Value Object»

Raiz de agregado = única porta de entrada para alterar o que está dentro dele; outros agregados são referenciados por ID

Identidade e Acesso

User
«Aggregate Root»

- Email «Value Object»
- Password «Value Object»

Invariantes: e-mail único; inativo não autentica; só o hash é salvo; link de acesso vale uma vez

Employee
«Aggregate Root»

Invariantes: papel ADMIN ou MECHANIC, alterado só por ADMIN; sempre resta um ADMIN

Clientes e Veículos

Customer
«Aggregate Root»

- Document «Value Object»
- Phone «Value Object»
- Email «Value Object»

Invariantes: documento válido e único; e-mail obrigatório (canal do primeiro acesso); login opcional

Vehicle
«Aggregate Root»

- LicensePlate «Value Object»

Invariantes: placa válida e única; pertence a um cliente (por ID)

Catálogo e Estoque

Service
«Aggregate Root»

- Money «Value Object»

Invariantes: preço ≥ 0; inativo não entra em orçamento novo

Product
«Aggregate Root»

- StockMovement «Entity»
- Money «Value Object»

Invariantes: saldo nunca negativo; todo saldo muda por movimentação

Atendimento

Order
«Aggregate Root»

Invariantes: RECEIVED → IN_PROGRESS → DELIVERED; deliver(...) só com todas as OS terminais

ServiceOrder
«Aggregate Root»

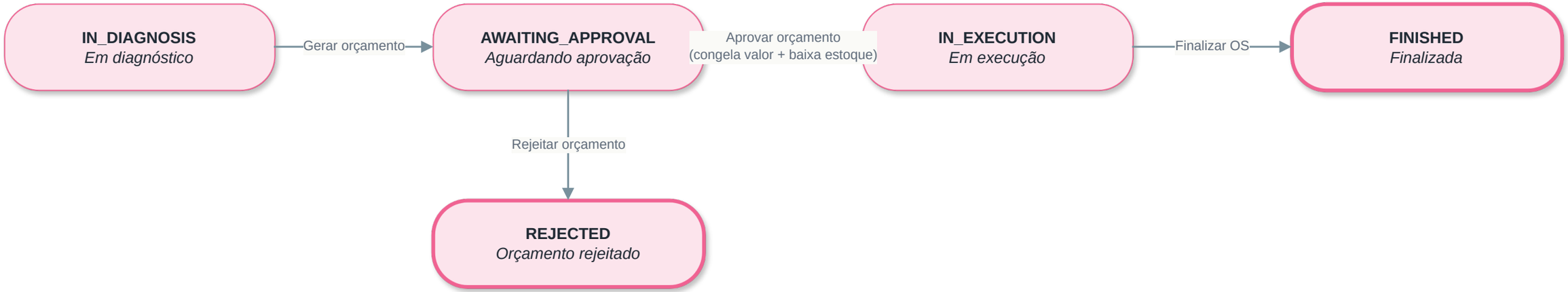
- ServiceItem «Entity»
- ProductItem «Entity»
- ServiceOrderStatusHistory «Entity»
- Money «Value Object»

Invariantes: transições só pelo fluxo; responsável é MECHANIC; itens só antes do orçamento; aprovar congela o valor; toda transição gera histórico

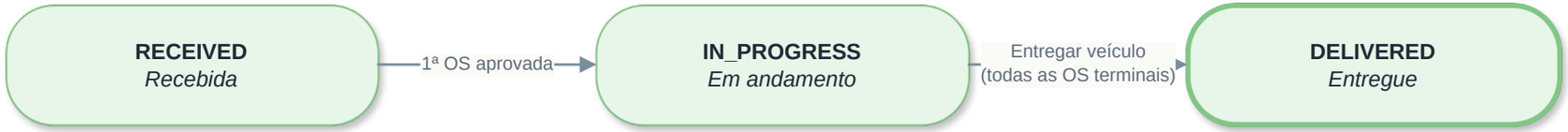
Máquinas de estado — ServiceOrder e Order

Borda grossa = estado terminal; toda transição da OS grava uma linha em ServiceOrderStatusHistory

ServiceOrderStatus · OS



OrderStatus · Ordem (estadia do veículo)



Fluxo do enunciado: Recebida (Order) → Em diagnóstico → Aguardando aprovação → Em execução → Finalizada (ServiceOrder) → Entregue (Order). O status da OS fica na ServiceOrder; o da estadia, na Order.